04-03. Window Service 구현 분석

개요

Window Service는 TVU 파이프라인에서 검증된 shred들을 수신하여 완전한 블록으로 재조립하고 Blockstore에 저장하는 핵심 컴포넌트입니다. 이 서비스는 누락된 shred의 복구, 중복 shred 탐지, 그리고 완성된 데이터 세트의 알림 등을 담당합니다.

WindowService 구조체

```
pub(crate) struct WindowService {
t_insert: JoinHandle<()>, // Shred 삽입 스레드
t_check_duplicate: JoinHandle<()>, // 중복 검사 스레드
repair_service: RepairService, // Repair 서비스
}
```

WindowService는 세 개의 주요 컴포넌트로 구성됩니다:

1. **Insert Thread**: Shred를 Blockstore에 삽입하는 메인 스레드
2. **Duplicate Check Thread**: 중복 shred를 탐지하고 처리하는 스레드
3. **Repair Service**: 누락된 shred를 복구하는 서비스

주요 데이터 구조

RepairMeta

```
struct RepairMeta {
nonce: Nonce, // Repair 요청 식별자
}
```

Repair를 통해 수신된 shred의 메타데이터를 저장합니다.

WindowServiceMetrics

```
#[derive(Default)]
struct WindowServiceMetrics {
run_insert_count: u64, // 삽입 실행 횟수
num_packets: usize, // 처리된 패킷 수
num_repairs: usize, // Repair 패킷 수
num_shreds_received: usize, // 수신된 shred 수
handle_packets_elapsed_us: u64, // 패킷 처리 시간
shred_receiver_elapsed_us: u64, // Shred 수신 시간
prune_shreds_elapsed_us: u64, // Shred 정리 시간
num_shreds_pruned_invalid_repair: usize, // 무효한 repair shred 수
num_errors: u64, // 총 오류 수
// ... 기타 오류 통계들
addrs: HashMap<SocketAddr, usize>, // 소스별 패킷 수
}
```

WindowService 초기화

```
pub(crate) fn  new(
blockstore: Arc<Blockstore>,
verified_receiver: Receiver<Vec<PacketBatch>>,
retransmit_sender: Sender<Vec<ShredPayload>>,
repair_socket: Arc<UdpSocket>,
ancestor_hashes_socket: Arc<UdpSocket>,
repair_quic_endpoint_sender: AsyncSender<LocalRequest>,
repair_quic_endpoint_response_sender: Sender<(SocketAddr, Vec<u8>)>,
exit: Arc<AtomicBool>,
repair_info: RepairInfo,
leader_schedule_cache: Arc<LeaderScheduleCache>,
verified_vote_receiver: VerifiedVoteReceiver,
completed_data_sets_sender: CompletedDataSetsSender,
duplicate_slots_sender: DuplicateSlotSender,
ancestor_hashes_replay_update_receiver: AncestorHashesReplayUpdateReceiver,
dumped_slots_receiver: DumpedSlotsReceiver,
popular_pruned_forks_sender: PopularPrunedForksSender,
outstanding_repair_requests: Arc<RwLock<OutstandingShredRepairs>>,
) -> WindowService
```

초기화 과정에서 다음 작업들을 수행합니다:

1. **Repair Service 생성**: 누락된 shred 복구를 위한 서비스 시작
2. **Duplicate Check Thread 시작**: 중복 shred 탐지 스레드 생성
3. **Insert Thread 시작**: 메인 shred 처리 스레드 생성

Wen Restart 모드 처리

```
// In wen_restart, we discard all shreds from Turbine and keep only those from r
// avoid new shreds make validator OOM before wen_restart is over.
let accept_repairs_only = repair_info.wen_restart_repair_slots.is_some();
```

Wen Restart 상황에서는 메모리 부족을 방지하기 위해 repair shred만 허용합니다.

Shred 삽입 파이프라인 (run_insert)

1. 패킷 수신 및 배치 처리

```
let mut shred_receiver_elapsed = Measure::start("shred_receiver_elapsed");
let mut packets = verified_receiver.recv_timeout(RECV_TIMEOUT)?;
packets.extend(verified_receiver.try_iter().flatten());
shred_receiver_elapsed.stop();
```

- 200ms 타임아웃으로 패킷 배치 수신
- 추가로 사용 가능한 모든 패킷 배치를 즉시 수집

2. 병렬 패킷 처리

```
let  handle_packet = |packet: &Packet| {
if  packet.meta().discard() {
return None;
}
let  shred = shred::layout::get_shred(packet)?;
let  shred = Shred::new_from_serialized_shred(shred.to_vec()).ok()?;
if  packet.meta().repair() {
let  repair_info = RepairMeta {
// If can't parse the nonce, dump the packet.
nonce: repair_response::nonce(packet)?,
};
Some((shred, Some(repair_info)))
} else {
Some((shred, None))
}
};


let (mut  shreds, mut  repair_infos): (Vec<_>, Vec<_>) = thread_pool.install(||
packets
.par_iter()
.flat_map_iter(|packets| packets.iter().filter_map(handle_packet))
.unzip()
```

```
});
```

Rayon 스레드 풀을 사용하여 병렬로 패킷을 처리합니다:

- 폐기 표시된 패킷 필터링
- 원시 패킷에서 Shred 객체 생성
- Repair 패킷의 경우 nonce 추출
- 일반 shred와 repair shred 분류

3. Repair 상태별 Shred 정리

```
fn prune_shreds_by_repair_status(
shreds: &mut Vec<Shred>,
repair_infos: &mut Vec<Option<RepairMeta>>,
outstanding_requests: &RwLock<OutstandingShredRepairs>,
accept_repairs_only: bool,
) {
let mut outstanding_requests = outstanding_requests.write().unwrap();
shreds.retain(|shred| {
let should_keep = (
(!accept_repairs_only || repair_infos[i].is_some())
&& verify_repair(&mut outstanding_requests, shred, &repair_infos[i]),
);
should_keep
});
```

```
}
```

다음 조건에 따라 shred를 필터링합니다:

- `accept_repairs_only` 모드에서는 repair shred만 허용
- 유효한 repair 응답인지 검증 (`verify_repair`)
- 미해결 repair 요청과 매칭되는지 확인

4. Blockstore 삽입 및 중복 처리

```
let  completed_data_sets = blockstore.insert_shreds_handle_duplicate(
shreds,
repairs,
Some(leader_schedule_cache),
false, // is_trusted
Some(retransmit_sender),
&handle_duplicate,
reed_solomon_cache,
metrics,
)?;
```

- Shred를 Blockstore에 삽입
- Reed-Solomon 오류 정정 코드를 사용한 데이터 복구
- 중복 shred 탐지 및 처리
- 완성된 데이터 세트 반환

중복 Shred 탐지 시스템 (run_check_duplicate)

중복 유형 분류

```
let (shred1, shred2) = match  shred {
PossibleDuplicateShred::LastIndexConflict(shred, conflict)
| PossibleDuplicateShred::ErasureConflict(shred, conflict) => {
if  send_index_and_erasure_conflicts {
(shred, conflict)
} else {
return Ok(());
}
}
PossibleDuplicateShred::MerkleRootConflict(shred, conflict) => {
if  merkle_conflict_duplicate_proofs {
// Store merkle conflict immediately
if  blockstore.has_duplicate_shreds_in_slot(shred_slot) {
return Ok(());
}
blockstore.store_duplicate_slot(
shred_slot,
conflict.clone(),
shred.clone().into_payload(),
```

```
)?;
(shred, conflict)
} else {
return Ok(());
}
}
PossibleDuplicateShred::Exists(shred) => {
// Check if duplicate already exists
if  blockstore.has_duplicate_shreds_in_slot(shred_slot) {
return Ok(());
}
let Some(existing_shred_payload) = blockstore.is_shred_duplicate(&shred) else {
return Ok(());
};
blockstore.store_duplicate_slot(
shred_slot,
existing_shred_payload.clone(),
shred.clone().into_payload(),
)?;
(shred, existing_shred_payload)
}
};
```

세 가지 중복 유형을 처리합니다:

1. **LastIndexConflict/ErasureConflict**: 인덱스나 erasure 충돌
2. **MerkleRootConflict**: Merkle 루트 충돌
3. **Exists**: 이미 존재하는 shred와의 중복

중복 증명 전파

```
// Propagate duplicate proof through gossip
cluster_info.push_duplicate_shred(&shred1, &shred2)?;
// Notify duplicate consensus state machine
duplicate_slots_sender.send(shred_slot)?;
```

중복이 탐지되면:

- Gossip 네트워크를 통해 중복 증명 전파
- 합의 상태 머신에 중복 슬롯 알림

Repair 검증 시스템

verify_repair 함수

```
fn verify_repair(
outstanding_requests: &mut OutstandingShredRepairs,
shred: &Shred,
repair_meta: &Option<RepairMeta>,
) -> bool {
```

```
repair_meta
.as_ref()
.map(|repair_meta| {
outstanding_requests
.register_response(
repair_meta.nonce,
shred,
solana_sdk::timing::timestamp(),
|_| (),
)
.is_some()
})
.unwrap_or(true)
}
```

Repair shred의 유효성을 검증합니다:

- Repair 메타데이터가 있는 경우 nonce로 요청 매칭
- 미해결 repair 요청과 일치하는지 확인
- 일반 shred는 항상 유효한 것으로 처리

스레드 구성 및 관리

Insert Thread (solWinInsert)

```
let thread_pool = rayon::ThreadPoolBuilder::new()
.num_threads(get_thread_count().min(8))
.thread_name(|i| format!("solWinInsert{i:02}"))
.build()
.unwrap();
```

- 최대 8개의 워커 스레드로 구성된 Rayon 스레드 풀
- 병렬 패킷 처리를 위한 전용 스레드 풀
- Reed-Solomon 캐시를 사용한 오류 정정

Duplicate Check Thread (solWinCheckDup)

```
Builder::new()
.name("solWinCheckDup".to_string())
.spawn(move || {
while !exit.load(Ordering::Relaxed) {
if let Err(e) = run_check_duplicate(/* ... */) {
if Self::should_exit_on_error(e, &handle_error) {
break;
}
}
}
})
```

```
.unwrap()
```

- 독립적인 중복 검사 스레드
- 200ms 타임아웃으로 중복 shred 수신
- 오류 발생 시 적절한 처리 및 복구

성능 모니터링 및 메트릭

메트릭 수집

WindowService는 다음과 같은 성능 지표를 수집합니다:

- **처리량 메트릭**: 패킷 수, shred 수, repair 수
- **지연시간 메트릭**: 패킷 처리 시간, shred 수신 시간
- **오류 메트릭**: 다양한 유형의 오류 발생 횟수
- **네트워크 메트릭**: 소스별 패킷 분포

메트릭 보고

```
if  last_print.elapsed().as_secs() > 2 {
metrics.report_metrics("blockstore-insert-shreds");
metrics = BlockstoreInsertionMetrics::default();
ws_metrics.report_metrics("recv-window-insert-shreds");
ws_metrics = WindowServiceMetrics::default();
last_print = Instant::now();
}
```

2초마다 메트릭을 보고하고 초기화합니다.

오류 처리 전략

should_exit_on_error 함수

```
fn should_exit_on_error<H>(e: Error, handle_error: &H) -> bool
where
H: Fn(),
{
match e {
Error::RecvTimeout(RecvTimeoutError::Disconnected) => true, // 채널 연결 끊김
Error::RecvTimeout(RecvTimeoutError::Timeout) => false, // 타임아웃 (계속)
Error::Send => true, // 송신 실패
_ => {
handle_error();
error!("thread {:?} error {:?}", thread::current().name(), e);
false // 기타 오류 (계속)
}
}
}
```

오류 유형에 따른 적절한 처리:

- 치명적 오류: 스레드 종료

- 일시적 오류: 계속 실행
- 기타 오류: 로깅 후 계속 실행

데이터 플로우

Window Service 내에서의 데이터 흐름:

1. **Shred 수신**: ShredFetchStage로부터 검증된 패킷 배치 수신
2. **병렬 처리**: Rayon 스레드 풀을 통한 패킷 파싱 및 shred 생성
3. **Repair 검증**: 미해결 repair 요청과의 매칭 검증
4. **Blockstore 삽입**: Reed-Solomon 복구와 함께 shred 저장
5. **중복 탐지**: 별도 스레드에서 중복 shred 탐지 및 처리
6. **완료 알림**: 완성된 데이터 세트를 ReplayStage에 알림

Window Service는 Solana의 고성능 블록 처리를 위한 핵심 컴포넌트로, 효율적인 shred 재조립과 강력한 오류 복구 메커니즘을 제공합니다.